

IA e o estetoscópio digital

Aplicação para diagnóstico automatizado¹



PARTE 1

Armazenamento e processamento local

Edge Computing

1

Introdução e fluxo de funcionamento

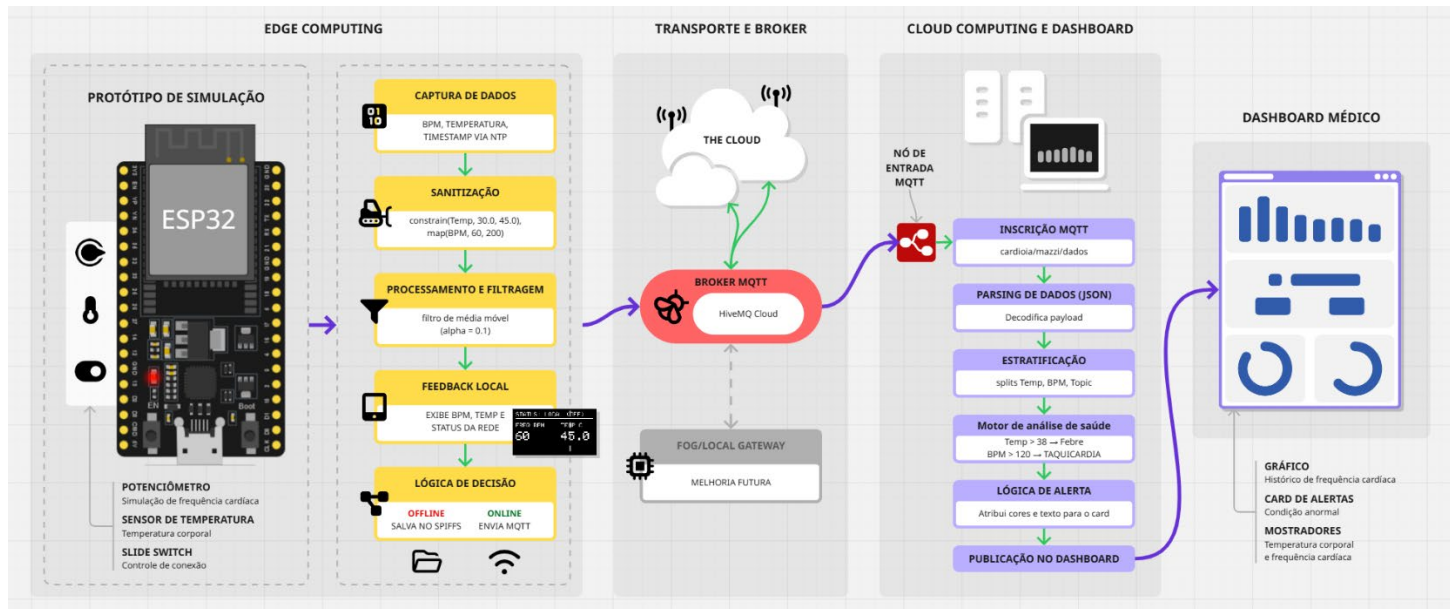
O objetivo desta etapa é demonstrar a inteligência na borda (Edge Computing) aplicada ao sistema CardiolA. O sistema utiliza um ESP32 para capturar dados de dois sensores: um DHT22 para temperatura corporal e um potenciômetro que simula a frequência cardíaca (BPM).

O fluxo de funcionamento segue o ciclo contínuo:

1. **Captura:** Leitura dos sinais analógicos e digitais a cada 5 segundos.
2. **Sanitização (Edge Data Cleaning):** Aplicação da função constrain para garantir que os dados fiquem entre 30°C e 45°C, evitando ruídos espúrios.
3. **Processamento:** Aplicação de um filtro de média móvel ($\alpha = 0.1$) para suavizar as oscilações de leitura.

¹ Para integrantes do grupo e professores, veja a seção **Créditos**, no final deste documento.

4. **Decisão de envio:** Verificação do estado do *Slide Switch* (simulando conectividade Wi-Fi).



Visão geral da arquitetura

O diagrama está dividido em três zonas de responsabilidade técnica:

- **Edge Computing**
Mostra os sensores físicos e os 5 passos cruciais que o ESP32 executa. Note como a Sanitização e o Processamento/Filtro ocorrem antes da decisão de envio. A ramificação "OFFLINE/ONLINE" mostra exatamente quando o SPIFFS é acionado como contingência e como o display local fornece IHC de baixa latência.
- **Network & Broker**
Ilustra a nuvem MQTT via HiveMQ Cloud. O Local Gateway é uma possível melhoria de arquitetura que discutimos para um futuro projeto real.
- **Cloud Computing e Dashboard**
Detalha o fluxo dentro do Node-RED. É importante notar que a Análise de Saúde e a Lógica de Alerta ocorrem sequencialmente, garantindo que o Dashboard visualize não apenas o número, mas o diagnóstico (veja o card vermelho de "Resposta Inflamatória Aguda" na figura Card de alerta dinâmico, na parte 2).

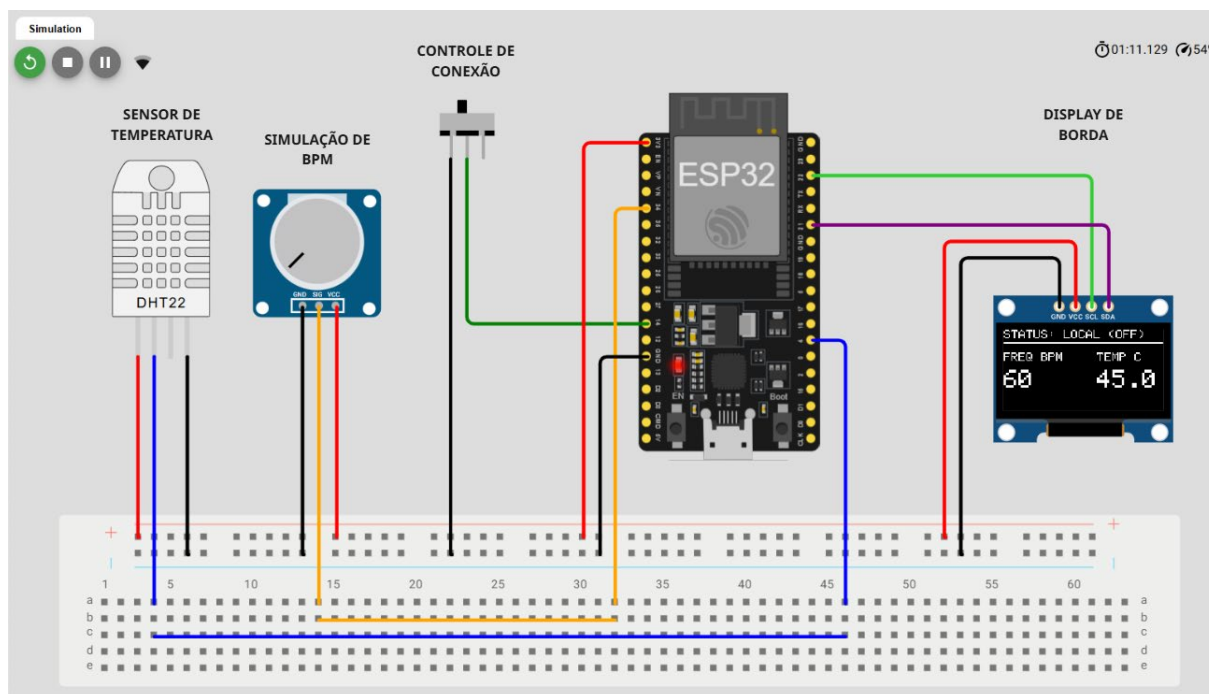
Lógica de resiliência offline

Para garantir que nenhum dado crítico seja perdido durante instabilidades de rede (resiliência), implementamos o uso da memória SPIFFS.

- **Modo Offline:** Quando o interruptor está em "LOW" (desconectado) (veja a figura Circuito de simulação adiante), o sistema gera um payload JSON completo, incluindo o Timestamp (TS) real via NTP, e o anexa ao arquivo backup.txt na memória Flash do ESP32.
- **Sincronização:** Ao detectar o retorno da conexão (Switch em "HIGH"), a função sincronizar() realiza a leitura sequencial do arquivo de backup e publica os dados retroativos no Broker MQTT.
Cabe ressaltar que empregamos uma **contingência ativa**, ou seja, mesmo com a ordem de

estar online (Switch HIGH), o código monitora o handshake do protocolo MQTT. Caso ocorra um time-out ou perda de pacotes, a lógica desvia automaticamente o dado para a memória não volátil (SPIFFS).

- **Eficiência de Armazenamento:** Após uma avaliação criteriosa (veja tabela mais abaixo), dimensionamos o sistema para suportar aproximadamente 30 horas de “apagão” em um chip físico real (1MB de SPIFFS), garantindo a integridade do histórico do paciente.



Circuito de simulação

Cenários de utilização

CENÁRIO	1. SIMULAÇÃO WOKWI (FIAP)	2. PROTÓTIPO FÍSICO (ESP32)	3. MUNDO REAL (ESCALA)
MEMÓRIA SPIFFS	~1.5 MB (Padrão)	1.0 MB (Particionado)	4.0 MB a 16.0 MB (Flash Externa)
TAMANHO DO REGISTRO	~45 Bytes	~45 Bytes	~30 Bytes (Binário/Protocol Buffers)
FREQ. DE COLETA	5 segundos	5 a 10 segundos	1 a 60 segundos (Dinâmico*)
CAPACIDADE REGISTROS	~33.000 registros	~22.000 registros	~130.000 registros (p/ 4MB)
TEMPO DE APAGÃO	~46 Horas	~30 Horas	~90 Dias (se coleta 1 registro / min)

Simulador vs hardware real

Embora o enunciado destaque que o SPIFFS é volátil em simuladores, o código foi escrito de forma "production-ready". No monitor serial, é possível observar a confirmação da gravação local, validando a lógica que seria executada em um dispositivo médico físico.

```
ets Jul 29 2019 12:21:46

rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:2
load:0x3fff0030,len:1156
load:0x40078000,len:11456
ho 0 tail 12 room 4
load:0x40080400,len:2972
entry 0x400805dc
E (740) esp_core_dump_flash: No core dump partition found!
E (80) SPIFFS: mount failed, -10025
Offline: {"temp":45.0,"bpm":60,"ts":11}
Dado guardado no SPIFFS.
Offline: {"temp":45.0,"bpm":60,"ts":16}
Dado guardado no SPIFFS.
Conexao restabelecida! Sincronizando...
Sincronizacao concluida.
Online: {"temp":45.0,"bpm":60,"ts":1778616153}
Offline: {"temp":45.0,"bpm":60,"ts":1778616158}
Dado guardado no SPIFFS.
Offline: {"temp":45.0,"bpm":60,"ts":1778616163}
Dado guardado no SPIFFS.
Online: {"temp":45.0,"bpm":60,"ts":1778616168}
Online: {"temp":45.0,"bpm":60,"ts":1778616173}
```

Saída do serial monitor do Wokwi em tempo de simulação, evidenciando a dinâmica de resiliência da solução

PARTE 2

Transmissão para Nuvem e Visualização Fog/Cloud Computing

Fluxo de Comunicação MQTT

A transmissão dos dados do ESP32 para a nuvem utiliza o protocolo **MQTT**, escolhido por sua leveza e baixo consumo de energia.

- **Broker**
Utilizou-se o HiveMQ Cloud como intermediário.

- **Payload**

As informações são encapsuladas em formato JSON, contendo temp, bpm e ts (Unix Timestamp).

- **Sincronismo de tempo**

Para evitar erros de cronologia, o sistema utiliza o horário de Brasília (GMT-3). Identificou-se, durante a simulação, um desafio de *Clock Drift* (deriva temporal) inerente ao processamento JavaScript do simulador, o que foi documentado como uma limitação externa, mantendo-se a fidelidade ao timestamp original do sensor para fins de auditoria.

- **Sincronismo de tempo e latência da rede**

Durante a fase de testes, observamos uma discrepância temporal (offset) entre a geração do dado e sua plotagem. A partir de uma análise técnica, identificamos que este fenômeno não possui uma causa única, mas pode ser o resultado de uma combinação de fatores sistêmicos, como, por exemplo:

1. **Clock drift de emulação:** A execução do código C++ dentro de um motor JavaScript (Wokwi) pode introduzir micro-atrasos no processamento de milissegundos, causando uma deriva natural do relógio interno do ESP32.
2. **Ordem de inicialização e handshake NTP:** A ordem em que o serviço Node-RED e o simulador são disparados interfere na janela de sincronização. Se o envio de mensagens MQTT precede a estabilização completa do protocolo NTP no dispositivo, os primeiros registros são gerados com timestamps inconsistentes, criando um "atraso percebido" no histórico do gráfico.
3. **Latência de propagação (middleware):** O tempo de trânsito do dado entre a borda, o Broker (HiveMQ) e o terminal local onde roda o Node-RED adiciona camadas de latência que, em sistemas de simulação, tornam-se mais evidentes.

Por outro lado, essa variação manteve um valor constante ao longo de cada seção de simulação e teste.

Apesar desses fatores, optou-se por manter a fidelidade ao **timestamp de origem**. Esta escolha prioriza a rastreabilidade médica: em um cenário real, é mais seguro saber exatamente *quando* o evento ocorreu no corpo do paciente (Edge), mesmo que a visualização na nuvem apresente um atraso de entrega, do que mascarar latências de rede utilizando o horário do servidor.

Configuração do dashboard Node-RED

O dashboard foi projetado focando na **Persona Profissional de Saúde** e na **Persona Paciente**, priorizando a clareza visual e o tempo de resposta.

- **Gráfico de Frequência Cardíaca:** Exibe a variação do BPM em tempo real, permitindo identificar arritmias ou picos de esforço.
- **Gauges de Monitoramento:** Medidores circulares para Temperatura e BPM que oferecem uma leitura rápida do estado atual do paciente.
- **Interface de Borda:** Adicionalmente, implementou-se um display local, garantindo que o usuário tenha acesso aos dados mesmo sem um computador por perto (feedback de baixa latência).



Display de borda

Dashboard 1.0

A opção pelo uso do Node-RED Dashboard 1.0 em detrimento da versão 2.0 (Dashboard 2.0 / Next-gen) fundamentou-se na maturidade e na estabilidade da biblioteca para o ambiente de prototipagem acadêmica e profissional. O Dashboard 1.0 possui uma documentação extensiva e uma integração nativa consolidada com os nós de processamento utilizados no projeto, garantindo menor latência na renderização dos gráficos de sinais vitais e maior confiabilidade na execução de scripts customizados (CSS/HTML). Além disso, a versão 1.0 mostrou-se mais resiliente em ambientes de execução local e simulada, assegurando que a experiência do usuário (UX) e a visualização das métricas de saúde fossem entregues sem as instabilidades típicas de versões em estágio de transição tecnológica.

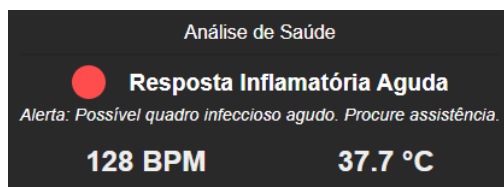
Alertas inteligentes

Implementamos um nó de **análise de saúde** que atua como um sistema de suporte à decisão. Fizemos uma breve pesquisa e criamos, para fins meramente didáticos e ilustrativos, uma tabela (veja a seguir) que relaciona as duas grandezas medidas (taxa de batimentos cardíacos e temperatura corporal) e atribui possíveis causas.

Ainda que estejamos lidando com uma relação direta entre grandezas, essa informação, numa aplicação real, poderia ser utilizada para treinar ou mesmo ser submetida a modelos inteligentes para diagnósticos reais ou aperfeiçoamento de modelos diagnósticos.

CONDIÇÃO	BPM BAIXO (<60)	BPM MÉDIO (60-100)	BPM ALTO (>100)
Temp. Baixa (<35°C)	Risco de Choque/Hipotermia	Hipotermia Leve	Resposta Metabólica Anormal
Temp. Média (35-37.5°C)	Bradicardia	Estado Estável	Taquicardia
Temp. Alta (>37.5°C)	Infecção Grave / Sepses	Estado Febril	Resposta Inflamatória Aguda

Indicador visual: um card dinâmico no Node-RED altera sua cor (Verde/Laranja/Vermelho) e exibe orientações médicas automáticas baseadas nos valores capturados.



Card de alerta dinâmico

Eficiência, segurança e aplicabilidade de mercado

A solução proposta é 100% aplicável em projetos reais de **Saúde Digital** e **Home Care**. Sua eficiência é garantida pelo processamento distribuído (Edge/Cloud), reduzindo o tráfego de dados desnecessários. A segurança é reforçada pela resiliência offline, garantindo que um dado perdido não se torne um diagnóstico errado. Boas práticas de IoT médico foram seguidas ao sanitizar os dados na origem e ao fornecer interfaces claras e acessíveis para o monitoramento contínuo.

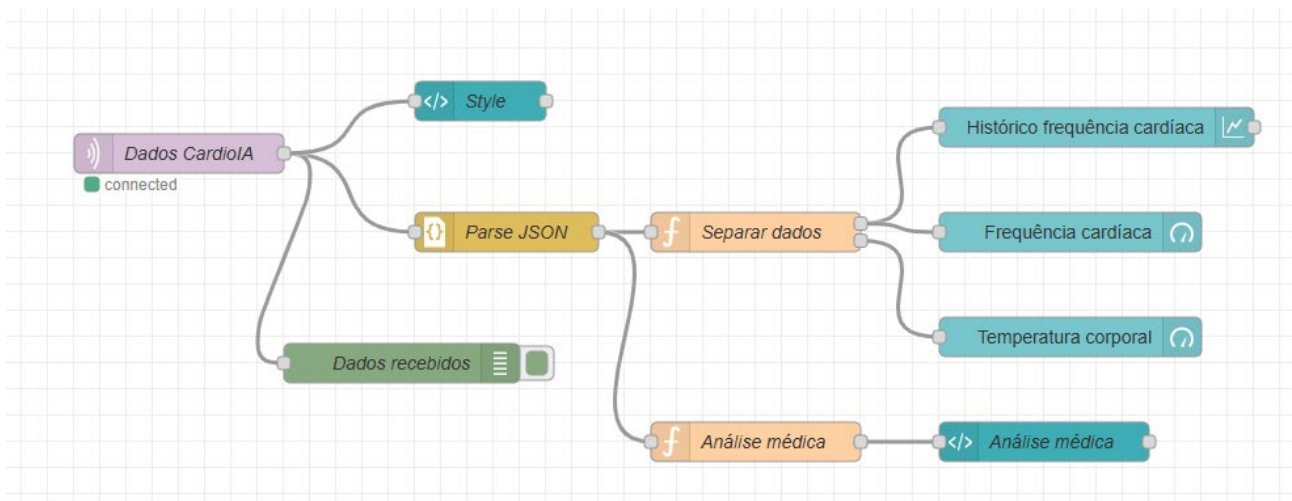
Entrega de valor

Numa situação real, podemos considerar diferentes cenários de uso para essa aplicação. A tabela a seguir, relaciona algumas possíveis personas e o valor de que cada uma delas pode se beneficiar.

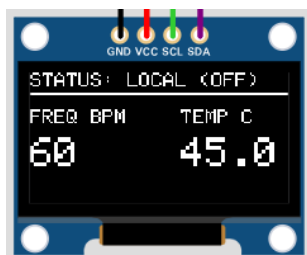
PERSONA	VALOR ENTREGUE (DORES RESOLVIDAS)
Paciente	Segurança e liberdade Saber que está sendo monitorado mesmo sem Wi-Fi (resiliência).
First Responders	Agilidade Receber um alerta de “Resposta Inflamatória” antes mesmo de chegar ao local, já sabendo o histórico imediato.
Profissionais de Saúde	Dados limpos e confiáveis O gráfico reconstrói o "apagão", evitando lacunas no diagnóstico.
Equipes de IA e TI	Escalabilidade A arquitetura MQTT permite monitorar 1 ou 10.000 pacientes com o mesmo back-end.

Evidências

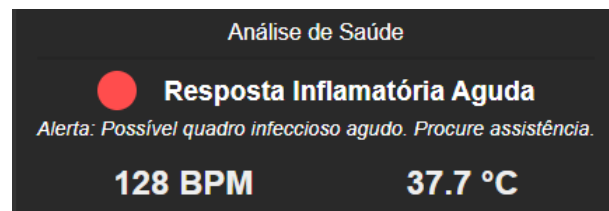
Seguem imagens capturadas da aplicação em seus diversos componentes.



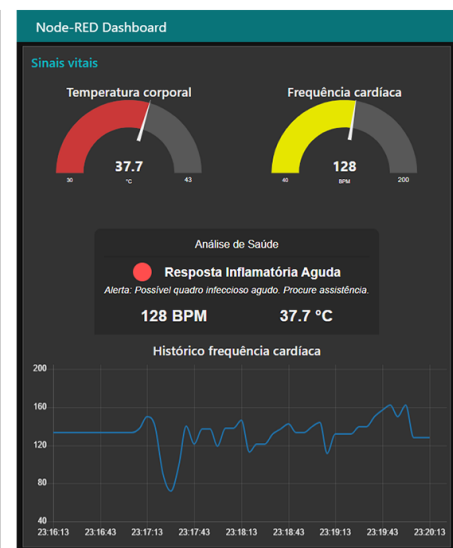
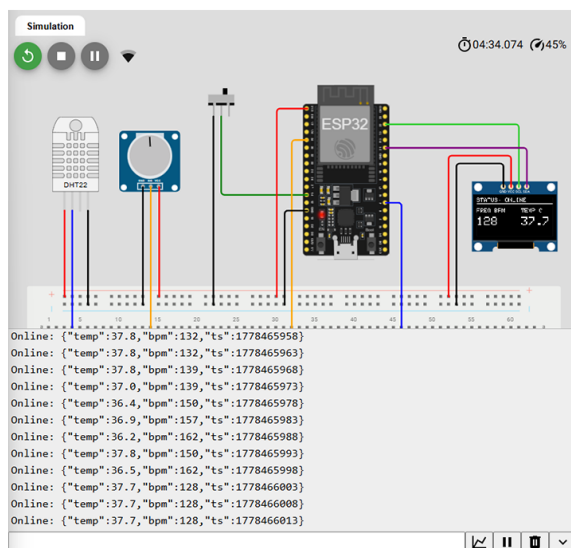
Configuração do Node-RED



Detalhe do display de borda



Detalhes do card de alertas do dashboard



Detalhes da simulação do Wokwi, do debug do Node_RED e do dashboard em tempo de simulação.

Entregáveis

Optamos por consolidar os relatórios das partes 1 e 2 em um único arquivo, pois há vários elementos, como screenshots e figuras, que dizem respeito a ambas e, assim, evitamos ter que repeti-las desnecessariamente.

Segue a relação de entregáveis das partes 1 e 2, além deste relatório:

PARTE 1	
Link do projeto no Wokwi	https://wokwi.com/projects/462930038877192193
Código em C++ comentado	Veja abaixo
Relatório descrevendo o fluxo de funcionamento e a lógica de resiliência	Este documento

PARTE 2	
Código do ESP32	Veja abaixo
Prints evidenciando a aplicação ou export do dashboard	Veja seção Evidências neste documento
Relatório sobre o fluxo de comunicação MQTT e configuração do dashboard	Este documento

Adicionalmente, este relatório, o export dos projetos Wokwi e Node-RED, assim como o código utilizado na solução, encontram-se no repositório

https://github.com/jayrom/FIAP_ia_med_app.git, phase 3.

Código C++ comentado

Apesar de ser parte do projeto WOkwi, segue abaixo o código, já que se trata de um requisito explícito.

```
#include <WiFi.h>
#include <PubSubClient.h>
#include "DHT.h"
#include "SPIFFS.h"
#include "time.h"
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

// Definições de hardware e constantes do display
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

// Mapeamento de pinos e instanciamento do sensor de temperatura
#define DHTPIN 4
#define DHTTYPE DHT22
#define POT_PIN 34
```

```
#define SWITCH_PIN 14
DHT dht(DHTPIN, DHTTYPE);

// Configurações de rede e endereçamento do Broker MQTT
const char* ssid = "Wokwi-GUEST";
const char* password = "";
const char* mqtt_server = "broker.hivemq.com";

WiFiClient espClient;
PubSubClient client(espClient);

unsigned long lastMillis = 0;
const long interval = 5000; // Intervalo de 5s para equilíbrio entre fluidez e tráfego

float alpha = 0.1; // Fator de suavização para o filtro de média móvel
float filteredTemp = 0;

// Parâmetros para sincronização de tempo real via servidor NTP
const char* ntpServer = "br.pool.ntp.org";
const long   gmtOffset_sec = -10800;           // GMT-3 (São Paulo)
const int     daylightOffset_sec = 0;

// Atualiza a interface do display OLED com hierarquia visual de dados
void atualizarDisplay(float t, int b, bool conectado) {
    display.clearDisplay();
    display.setTextCursor(0, 0);
    display.setTextColor(SSD1306_WHITE);

    // Exibe o status da conexão para feedback imediato ao usuário
    display.setTextSize(1);
    display.setCursor(0, 0);
    if (conectado) {
        display.print("STATUS: ONLINE");
    } else {
        display.print("STATUS: LOCAL (OFF)");
    }
    display.drawLine(0, 10, 128, 10, SSD1306_WHITE);

    // Rótulos dos sinais vitais (UX: facilitação da leitura rápida)
    display.setTextSize(1);
    display.setCursor(0, 18); display.print("FREQ BPM");
    display.setCursor(75, 18); display.print("TEMP C");

    // Valores numéricos em destaque para prioridade de informação
    display.setTextSize(2);
    display.setCursor(0, 32); display.print(b);
    display.setCursor(75, 32); display.print(t, 1);

    display.display();
}

// Armazena payloads na memória Flash (SPIFFS) durante quedas de conexão
void gravarLocal(String payload) {
    File file = SPIFFS.open("/backup.txt", FILE_APPEND);
    if (file) {
        file.println(payload);
        file.close();
        Serial.println("Dado guardado no SPIFFS.");
    }
}

// Recupera e transmite dados acumulados após o restabelecimento da rede
void sincronizar() {
    if (SPIFFS.exists("/backup.txt")) {
        Serial.println("Conexao restabelecida! Sincronizando...");
        File file = SPIFFS.open("/backup.txt", FILE_READ);
    }
}
```

```

while (file.available()) {
    String linha = file.readStringUntil('\n');
    client.publish("cardioia/mazzi/dados", linha.c_str());
    delay(500);
}
file.close();
SPIFFS.remove("/backup.txt");
Serial.println("Sincronizacao concluida.");
}
}

// Inicialização de periféricos, sistema de arquivos e serviços de rede
void setup() {
    Serial.begin(115200);
    pinMode(SWITCH_PIN, INPUT_PULLUP);

    // Inicializa OLED
    if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
        Serial.println("Erro: SSD1306 nao alocado");
    }
    display.clearDisplay();
    display.display();

    if (!SPIFFS.begin(true)) {
        Serial.println("ERRO ao montar SPIFFS.");
    }

    setup_wifi();
    client.setServer(mqtt_server, 1883);
    dht.begin();
    configTime(gmtOffset_sec, daylightOffset_sec, ntpServer);
}

// Gerencia a conexão com o ponto de acesso Wi-Fi
void setup_wifi() {
    delay(10);
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) { delay(500); }
}

// Tenta reconectar ao Broker MQTT se o switch de rede estiver ativo
void reconnect() {
    if (digitalRead(SWITCH_PIN) == HIGH && !client.connected()) {
        if (client.connect("ESP32_Mazzi_CardioIA")) {
            sincronizar();
        }
    }
}

// Ciclo principal: gerencia conectividade, processa sensores e envia dados
void loop() {
    bool wifiForcado = (digitalRead(SWITCH_PIN) == HIGH);

    if (wifiForcado) {
        if (!client.connected()) {
            static unsigned long lastReconnectAttempt = 0;
            if (millis() - lastReconnectAttempt > 5000) {
                lastReconnectAttempt = millis();
                reconnect();
            }
        }
        client.loop();
    }

    // Processamento Edge: Leitura, Sanitização (Clamping) e Filtragem

```

```

float rawTemp = dht.readTemperature();
if (isnan(rawTemp)) {
    rawTemp = filteredTemp;
} else {
    rawTemp = constrain(rawTemp, 30.0, 45.0);
}

// Mapeia sinal analógico para batimentos cardíacos simulados
int rawBpm = map(analogRead(POT_PIN), 0, 4095, 60, 200);

// Algoritmo de filtragem para eliminar ruídos de leitura do sensor
if (filteredTemp == 0) filteredTemp = rawTemp;
filteredTemp = (alpha * rawTemp) + ((1.0 - alpha) * filteredTemp);

// Timer para transmissão periódica e atualização do feedback local
if (millis() - lastMillis >= interval) {
    lastMillis = millis();

    time_t now;
    time(&now);

    // Atualiza o Display Local (Feedback imediato)
    atualizarDisplay(filteredTemp, rawBpm, client.connected());

    String payload = "{\"temp\":\"" + String(filteredTemp, 1) +
        "\",\"bpm\":\"" + String(rawBpm) +
        "\",\"ts\":\"" + String(now) + "\"}";

    // Decisão de envio: Transmissão direta ou armazenamento local (contingência)
    if (wifiForcado && client.connected()) {
        if (client.publish("cardioia/mazzi/dados", payload.c_str())) {
            Serial.println("Online: " + payload);
        } else {
            gravarLocal(payload);
        }
    } else {
        Serial.println("Offline: " + payload);
        gravarLocal(payload);
    }
}
}
}

```

Créditos

Integrantes

- [RM565912 - Edmilson Marciano](#)
- [RM565576 - Jayro Mazzi Junior](#)
- [RM563353 - Lucas Arcanjo](#)

Professores

Tutor

- [Caique Nonato da Silva Bezerra](#)

Coordenador(a)

- [Andre Godoi Chiovato](#)